

MERCEDES-BENZ · AIOS · MVA

Design Sync

MVA — product strategy, the assistant's form, and generative UI for I3 MAX.

PRODUCT × DESIGN • I3 MAX • FOR REVIEW

SCROLL

01

Overall MVA product strategy

MVA is moving from a voice assistant to the intelligence layer of the whole cabin — it converses, acts, remembers, and orchestrates. Seven directions:

01 Natural full-duplex dialogue

LLM-driven, free and human-like: broader context, a more natural voice, stronger false-trigger rejection.

02 Collaborative task completion

A GUI agent that breaks a spoken request into steps and runs them itself: find, download, open an app, jump to the exact page.

03 Cross-app orchestration

Dispatches and combines the cabin's atomic capabilities across app boundaries, delivering the service or content directly instead of making the user app-hop.

04 Memory & personalization

Long- and short-term memory of behavior and preference; proactive prompts timed from real usage data, not guesses.

05 Adaptive form

Different forms across interfaces and hardware, reshaping to the scene and generating the content that fits.

06 Multimodal perception

Senses and reflects the world back (vision, text, audio, cabin state, fused through the LLM) for more direct feedback and mutual trust.

07 Expert orchestration

Marshals domain expert agents, hands them tasks, and turns their expertise into one continuous conversation with the user.

Direction 5 — adaptive form — turns straight into a design question: what should that intelligence actually look like on screen?

↓ TOPIC 02

02

The MVA “avatar” for I3 MAX

As the visual representative of AIOS, MVA should be an **abstract form** — like **Siri and Gemini** — not a solid face or character. Our current Starlight (星辉) and little-Benz (小奔) are still too figurative; for I3 MAX we want to move past them to something fully abstract. Whether that abstraction is light, an orb, or something else is open — the point is abstract, not a character.

Starting from I2 MAX, we want the whole system to re-skin for holidays and campaigns, Tesla-style — avatar included. Once you commit to theming, the form has to stay abstract to be more flexible. Theming is one of the reasons to keep it abstract rather than a fixed character.

◆ WHY ABSTRACT WINS · THEMING — THE STRONGEST ARGUMENT

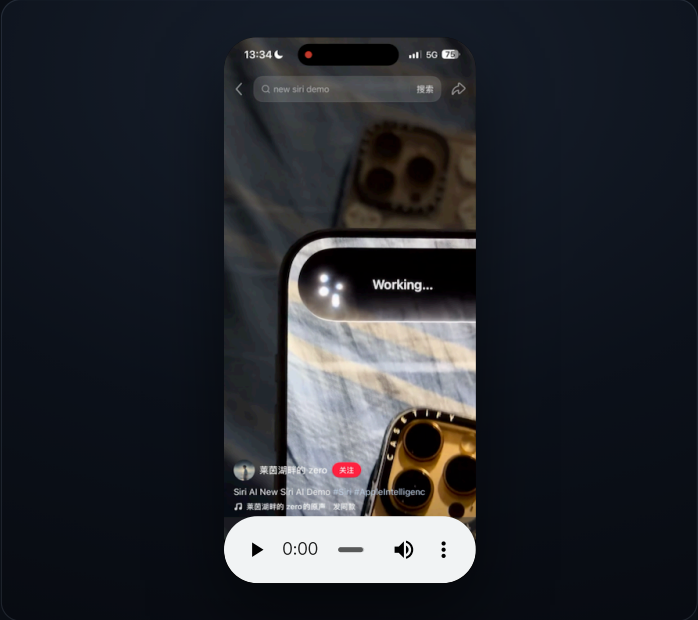
Parametric, not an asset. An abstract form is a parameter set (color, motion, density) and a theme is just a preset. A character is a fixed asset — re-skinning means redraw + re-rig every time, or a hat slapped on while the character fights the theme underneath.

The identity paradox. A character's value is a stable, recognizable identity — but theming demands change, so the more you alter it the more you erode what justified it. An abstract form has no fixed identity to erode, so it re-skins infinitely — which is exactly what our planned celebrity-customized personas need: a base you can dress in a star's voice and visual signature, swap per user, and retire on demand. A fixed character can only ever be itself; an abstract presence can wear any star's cues — signature, palette, motion — e.g. 张艺兴, 苏翊鸣, 王楚钦.

Coherence & cadence. An abstract form lives in the system's own material, so it themes in lockstep and ships at token speed — a dozen skins a year. A character is a foreign object, hand-matched per theme, two a year at best.

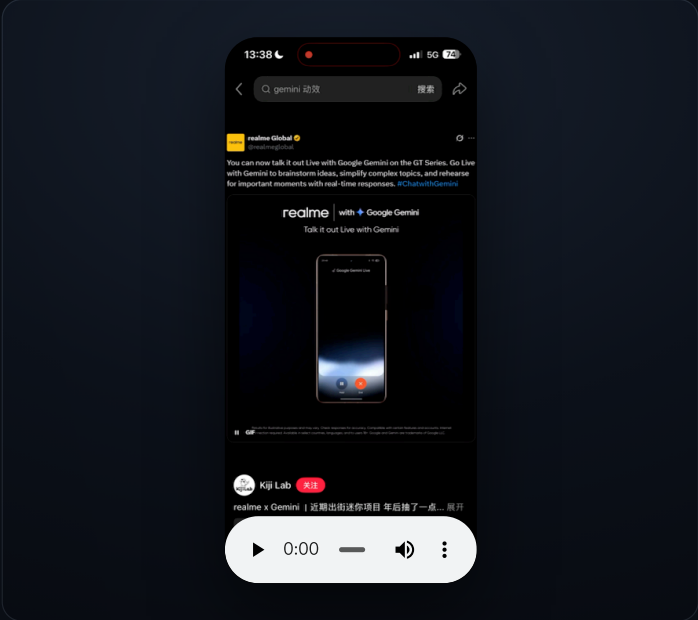
The industry already turned. Tesla — most committed to whole-system theming — has no avatar; its personality lives in motion, sound, and light. Siri, Gemini, ChatGPT, and Super Eva are all moving their assistants toward an abstract form.

◆ WHAT “ABSTRACT” LOOKS LIKE IN THE WILD



Apple Intelligence · Siri

the screen-edge glow, in use



Google · Gemini

Gemini Live, in use

The assistants in use — presence rendered as light and gradient, not a character.

◆ SUPPORTING REASONS

Positioning. An abstract presence reads as “the whole system is responding to you” — the OS-level register. A character demotes it to “a thing living inside the screen.”

Craft goes up. The work shifts from drawing a character to designing a living material and a motion system.

Warmth through behavior, not a face. Personality lives in how it breathes, anticipates, and responds — in motion, voice, and tone.

The Nebula window and the smart island beneath it are already the AI's body and status display — they carry activity, results, and state. A heavy, shape-shifting character on top spends the expressiveness budget twice and competes for the same attention. Cleaner split: the window and island carry the **what**; MVA only needs to signal **alive / listening** — one quiet, ambient cue, readable in peripheral vision, no extra focal point for the driver.

Restraint here isn't compromise, it's maturity.

■ ALL IN ALL — WHAT WE NEED FOR I3 MAX

1 An abstract avatar + Nebula window + smart island, designed as one

smart island = the former smart bar

- One shared style and ambient-effect system.
- Cohesive motion across all three — for example:
 - the avatar's expression reflects what the Nebula window is showing;
 - an AI recommendation surfaces in the smart island, then resolves into a card in the feed.

2 MVA lives inside the overall system design — not beside it or added on to it

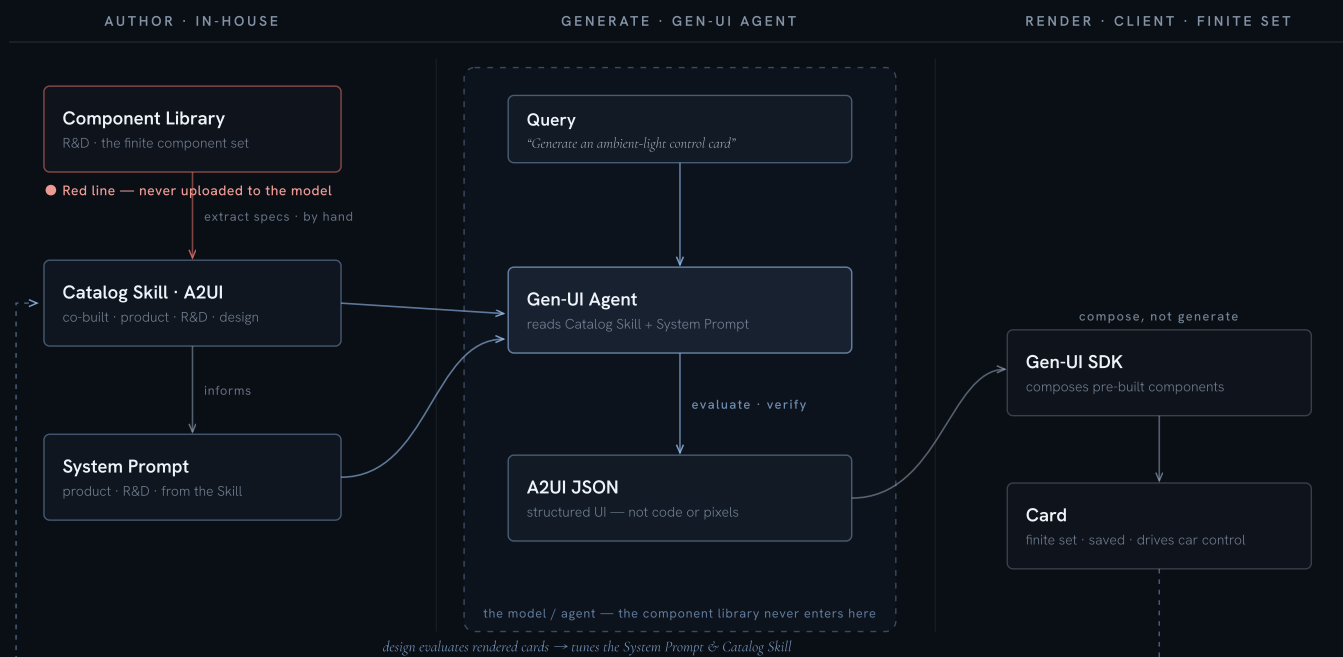
- **Onboarding** — the avatar is present from first run.
- **Profile** — MVA customization ties to the user profile; MVA is exclusive per user, including an MVA “memory board” in the profile module.
- **Ambient language** — how MVA expresses itself across modules:
 - empowering any module (e.g. image optimization);
 - execution — a “say what you see” execution animation;
 - AI recommendation.

That ambient language is really Gen-UI applied across the system — which is where we'd like your input.

That ambient language is really Gen-UI applied across the system. If we generate components, here's the pipeline — and where design sits in it.

Gen-UI & widgets for I3 MAX

Whatever design decides, the bar is the same: across two surfaces — chat UI and vehicle-control components — we need to satisfy diverse, scene-specific visual needs as flexibly as possible (the scenario list is tracked separately). If design chooses to generate some of those components with Gen-UI, this is the flow — and the two places design holds the wheel.



The pipeline: authored in-house, generated as structured A2UI JSON, composed from a finite component set on the client — with design closing the loop.

◆ HOW TO READ IT — FOUR THINGS TO HOLD ONTO

RED LINE The component library never goes to the model. It's built for R&D and is the finite set of real components. Uploading it to the model does not happen — this is the hard line. The model only ever sees the Catalog Skill and the System Prompt.

The Catalog Skill is the quality gate. A document co-built by product · R&D · design, in Google's A2UI form — it lifts the component library's design specs and cautions into a per-component schema the model can read. Generation quality lives or dies here.

The System Prompt is the second lever. Written by product · R&D from the Skill — it also shapes what the model is allowed to produce.

The SDK composes, it doesn't generate. R&D's Gen-UI SDK assembles pre-built components per scene from a finite set — composition, not free-form front-end. Because the set is bounded, design can evaluate every result and feed corrections back into the prompt and the Catalog Skill.

Design holds the wheel at both ends — authoring the Catalog Skill that bounds what can be built, and judging the cards that tune it. Nothing is rendered live on the device: cards are composed from pre-built components, their JSON generated and saved back-stage — and every step of that pre-build is bounded by design.

NEXT STEP — THE ASK

Product · R&D and design run this pipeline together — one joint pass, end to end.

- 01 Is the **flow feasible** in practice — does it hold together end to end?
- 02 Do the **generated results** clear **design's quality bar**?



MERCEDES-BENZ

AIOS · MVA · DESIGN SYNC · INTERNAL — FOR REVIEW